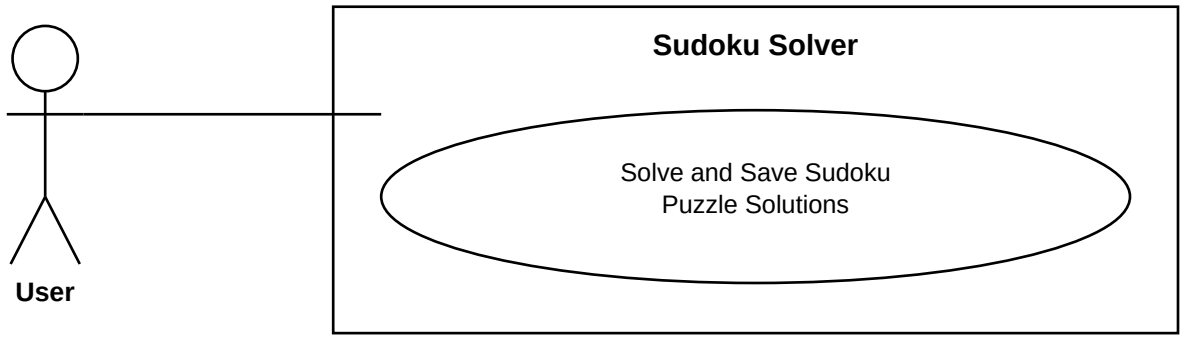
EECS 348 Assignment 5 - UML Use Case: Solve and Save Sudoku Puzzle Solutions

Revision of Assignment 4's use case ("Solve Sudoku Puzzles," originally authored by Andy Garcia with drafting help from Claude (claude.ai)). This revision is by Lucas Frias for Assignment 5, updated with assistance from Claude (claude.ai) and reviewed against modified.cpp by the author.

UML Use Case Diagram



1. Identify / Discover the Use Case

Use case: **Solve and Save Sudoku Puzzle Solutions** . Actor: **User** (the person who runs the program). Goal of value to the actor: for each of the five puzzle files, end up with a solutionN.txt file on disk containing every solution to that puzzle, or a clear statement that none exists, without having to read or save the solutions from the console output.

2. Brief Description

This use case allows a User to run the program so that it reads five Sudoku puzzle files and finds every solution for each one. For each puzzle, the system writes a corresponding solutionN.txt file containing either all solutions found or a "No solution found" message, in the same cell format as the original puzzle files. The console is used only to report short status messages (which file is being processed, errors, and a one-line summary pointing to the output file) — the full puzzle grid and solutions are no longer printed to the console.

3. Step-by-Step Flow (Outline)

1. Start program
2. Open puzzle file
3. Read and validate puzzle
4. Check given digits
5. Create solution output file
6. Search for solutions
7. Write and report results

4. Detailed Description

1. Start Program

The User runs the program from the directory containing puzzle1.txt through puzzle5.txt. The system processes the files in order, 1 through 5.

2. Open Puzzle File

The system prints the name of the current puzzle file to the console and tries to open it. If the file cannot be opened, the system prints "Error: Could not open file <filename>" to the console and moves on to the next puzzle file. No solution file is created for a puzzle that fails at this step.

3. Read and Validate Puzzle

The system reads 81 cells. Each cell must be a digit from 1 to 9 or an underscore (_) for a blank. If the file runs out of data or contains any other character, the system prints "Error: Invalid puzzle in <filename>" to the console and moves on to the next puzzle file. No solution file is created for a puzzle that fails at this step. Unlike Assignment 4, the system no longer displays the original 9x9 grid on the console at this stage — the only record of the puzzle going forward is the solution file produced in later steps.

4. Check Given Digits

The system checks that no starting digit repeats in its row, its column, or its 3x3 block. This check happens automatically as the first part of solving, before any output file is created. If a digit repeats, the puzzle cannot have a solution, so the search is skipped entirely and the system proceeds straight to writing and reporting a "No solution found" result.

5. Create Solution Output File

The system derives the output filename by replacing the "puzzle" prefix of the input filename with "solution" (e.g., puzzle3.txt → solution3.txt) and opens that file for writing. If the file cannot be created, the system prints "Error: Could not create file <solutionFilename>" to the console and moves on to the next puzzle file.

6. Search for Solutions

The system uses recursive depth-first search with backtracking. For each blank cell it tries the digits 1 through 9 that do not break a rule, moves to the next cell, and undoes the choice when it reaches a dead end. The search continues until every possibility has been tried, so all solutions are found. Each complete solution is written directly to the open solution file as soon as it is found, labeled "Solution N:" and followed by the solved 9x9 grid, rather than being printed to the console.

7. Write and Report Results

If no solution was found, the system writes "No solution found" to the solution file and prints "No solution found. See <solutionFilename>" to the console. Otherwise, the system writes a final "Solutions found: N" line to the solution file (N being the total count) and prints "Solutions found: N. See <solutionFilename>" to the console. The solution file is then closed, and the system moves on to the next puzzle file. After puzzle5.txt is processed, the program ends.